

A3PS — Logic & Pipeline Reference

The **why and how** of every file that matters to execution, followed by the end-to-end flow of the whole project including every change made in the latest round of work (no-render mode, per-stage latency, ablation switches, reactive baseline, matched-subset mTTA, threshold sweep, qualitative figure, permissive-prompt test). Companion to execution.md (how to run) and metrics.md (metric details).

Part 1 — The five pipeline phases and the logic inside each

Phase 1+2 fused: Perception + Tracking — a3ps/tracking/tracker.py

Key design decision: ONE model call does both. Tracker runs YOLOv8-Seg in **tracking mode** — `model.track(persist=True, tracker='botsort_a3ps.yaml')` — which returns detections, segmentation masks, AND BoT-SORT track IDs in a single inference pass. A separate Segmenter pass would double inference cost for identical information. (This is why per-stage latency reports perception: 0.0 — there is no separate perception call to time; the whole fused cost lands under tracking.)

Logic worth knowing:

- Class filtering happens AFTER tracking, not inside `model.track()`.
Passing `classes=` into the track call breaks BoT-SORT's association (observed: 10/540 frames got IDs with the filter vs ~300 without). So ALL classes are tracked for stable association, then non-target classes are dropped from the output.
- `centroid_img` is the bbox **foot point** (bottom-center), not the box center — it's the actor's ground-contact point, which is what a ground-plane homography can meaningfully project.
- `TrajectoryBuffer` decimates the 30 fps track stream into 5 Hz buckets (first sample per 0.2 s bucket), keeping the last 2 s (=10 points) per track ID. A track is `ready()` for forecasting once it has $\geq 80\%$ of that window. IDs unseen for 1 s are pruned.
- Device auto-detect: `.cuda().half()` when available, CPU fp32 otherwise.
- Tracker config `configs/botsort_a3ps.yaml` raises `track_buffer` 30→90 frames so brief occlusions/dropouts don't kill a track ID.

Phase 3: Forecasting — a3ps/forecasting/

`kalman_cv.py` (CORE, the production forecaster). Constant-velocity Kalman filter, state `[x, y, vx, vy]`:

1. **Fit**: run `predict+update` over the track's 10-point history.
 2. **Forecast**: roll `predict()` forward 20 steps (4 s @ 5 Hz) ***without updates***, so covariance — and the reported std — grows with each step.
- Reported std is the predicted measurement std `sqrt(diag(HPHT+R))`, not the raw state covariance: including R keeps step-0 std off the floor so growth over the horizon reads as a stable ~2.1x rather than an unbounded ratio. Tuning: `MEAS_VAR=1.0`, `PROCESS_VAR=0.1` (chosen by ablation; 4.0 gave ~7.9x std growth).
 - Histories under 4 points fall back to straight-line extrapolation with a fixed large std (40 px) — not enough data for a meaningful filter.

`seq2seq.py` (STRETCH, learned). Encoder-decoder LSTM (hidden 64, 2 layers): encodes the 10-point history, decodes 20 future **position deltas** plus a per-step log-variance head (so it has native uncertainty, trained

with Gaussian NLL). `predict()` applies the exact mining-time normalization (translate last-history point to origin, rotate mean heading to +y) on the way in and inverts it on the way out — making it a drop-in replacement for Kalman-CV behind the same `predict(history, dt, horizon_s)` interface.

Which one runs? `configs/default.yaml` → forecaster: `kalman_cv|seq2seq` (new ablation switch). The dashboard/pipeline default is Kalman-CV; the LSTM is primarily an eval/report artifact. Current result (289-window val set): LSTM wins at the 4 s horizon (ADE 56.33 vs 61.27 px, FDE 99.74 vs 122.55), Kalman marginally better at 1-2 s where motion is near-constant-velocity.

Phase 4: Risk + Decision — a3ps/risk/

`gaussian.py` — each forecast step is a diagonal 2D Gaussian (mean + per-axis std). `sigma_points_2d()` produces the 5 UKF sigma points (mean with weight 0 at $\kappa=0$, plus 4 points at $\pm\sqrt{2}\cdot\text{std}$ each weighted $\frac{1}{4}$).

`collision.py` — the probability math:

1. Per-step: collision probability = weighted fraction of the 5 sigma points that fall inside the ego corridor dilated by the actor's radius (Minkowski sum with a disk — a car "collides" when its body, not its center point, enters the path). Point-in-dilated-polygon = inside, OR within radius of the boundary (pure-Python signed-distance test, equivalent to `cv2.pointPolygonTest >= -radius` but dependency-free).
2. Per-trajectory: the 20 step probabilities are smoothed with a 3-step moving average; `max_prob` = the peak; `ttc_s` = time of the first step whose smoothed prob exceeds 0.5 (else the argmax step).
3. Cross-frame: RiskSmoothen EMA ($\alpha=0.4$) blends each frame's `max_prob` into a running per-track value, so a single-frame forecast glitch cannot fire an intervention. (Toggleable via `ema_smoothing`: false for the ablation.)

`decision.py` — DecisionEngine, the per-actor state machine:

- `active_threshold` = `base_threshold(0.75)` — $\Sigma(\text{context reductions})$, floored at 0.45. Reductions: `crosswalk_ahead` −0.10, `intersection` −0.10, `dense_traffic` (≥ 8 tracks in frame) −0.05. Crosswalk/intersection come from an optional hand-written `context.json` per clip (honestly NOT detected from imagery); `dense_traffic` is derived from perception. Toggleable via `dynamic_threshold`: false (ablation) → flat base threshold.
- Per actor: SAFE → ALERT → BRAKE. SAFE→ALERT when `prob` $\geq$ `threshold`−0.15 for 3 consecutive frames; ALERT→BRAKE when `prob` $\geq$ `threshold` for 3 consecutive frames. Each forward transition emits exactly ONE event — a near-miss yields one ALERT + one VIRTUAL_BRAKE, never one per frame.
- BRAKE is terminal per incident; the actor re-arms only after `prob` falls below 0.4 AND a 2 s cooldown passes. `THRESHOLD_LOWERED` (scene-level event) fires on the rising edge of an annotation span.

Phase 5: Explanation (XAI) — a3ps/explain/

`templates.py` (CORE, deterministic). Every emitted event gets a grammar-built one-liner with zero LLM involvement:

`"<Action>: <cls> ID-<id> <motion phrase> ego path in <ttc> s (P=<p>, threshold <t>[ — lowered due to <reason>])`" The motion phrase is inferred from BEV velocity vs the corridor (approaching head-on / cutting into / trajectory crossing / decelerating within). Deterministic = structurally faithful: it can only state facts the risk engine actually computed.

`llm_client.py` (STRETCH, offline). Optional enrichment that runs AFTER the pipeline, never in the live loop: per event, grab the keyframe from `raw.mp4`, downsize to 768 px, send structured facts + image to Groq's free-tier vision model, store the 2-3 sentence narrative in `explanation_llm`. The `SYSTEM_PROMPT` is facts-only-constrained ("Do not

invent objects or numbers not present in the facts"). Skips already-enriched events; --overwrite, --dry-run, retry-with-backoff. enrich_events() accepts a system_prompt override — existing solely so:

llm_client_permissive_test.py (TEMPORARY, illustration only) can reproduce a hallucination for the paper's before/after example: it runs the same events under the real prompt and under a deliberately unconstrained one, prints both side by side, then auto-restores the constrained narrative to disk so the hallucination-prone text never leaks into anything real.

The orchestrator — a3ps/pipeline.py

Pipeline(config, forecaster=?, risk_engine=?, explainer=?) wires the phases via three *injectable hooks* — each phase is optional and pluggable without editing the orchestrator. Per frame: track → fill BEV → build ctx → forecaster hook per track → risk hook per frame (which also stamps context.active_threshold for the dashboard) → attach the template explanation to each emitted event → record a FrameRecord → render.

Recent changes in this file:

- render: bool = True parameter on run() — when False, no VideoWriter, no annotated.mp4, no raw.mp4 copy; only events.json + meta.json. Batch eval over 120 clips gets much faster.
- Per-stage latency actually persists now. There was a real ordering bug: the timing dict was attached to result.meta *after* meta.json was written to disk, so the file never contained timing. Fixed — per_stage_ms (perception (0.0 by convention — see fused note above), tracking, forecasting, risk_decision, each ms/frame) plus an explanatory per_stage_ms_note are computed via the pure function build_per_stage_ms() and written before the save.

Shared foundations — a3ps/common/

schema.py — the single data contract everything speaks:

```
ClipResult { meta, frames[FrameRecord], events[Event] };
FrameRecord { frame_idx, t, tracks[TrackState], ego, context };
`TrackState { id, cls, bbox, centroid_img/bev, velocity_bev, mask_poly,
history_img, prediction, risk_level }`;
`Prediction { horizon_s, dt, mean_img, mean_bev, std_bev, collision_prob,
ttc_s }; Event { type, actor_id, collision_prob, threshold, ttc_s,
explanation_template, explanation_llm, ... }`. Every class round-trips
losslessly through JSON (to_dict/from_dict); optional BEV fields
serialize as null, never dropped. Because perception, tracking,
forecasting, risk, eval scripts AND the JS dashboard all read only this
schema, each layer is independently developable and testable.
Convention: while forecast_space: img, std_bev holds *pixel* stds —
the field name is kept stable so nothing downstream changes when BEV lands.
```

geometry.py — GroundPlane: homography between image points and BEV metres, built from 4 image points ↔ 4 ground points in the config (or a per-clip ground.yaml override). Also point_in_polygon. The default calibration is generic — which is exactly why the pipeline defaults to forecast_space: img until per-clip calibration (Week 4).

Part 2 — Scripts (entry points), and the logic inside each

scripts/prepare_nexar.py — dataset → index

Auto-detects and merges every labels table in data/nexar/ (matching videos by id, label from target), probes each video's metadata, computes splits — dev = 15 shortest (10 neg + 5 pos), eval = fixed 60+60, every remaining negative → train_traj — writes index.csv, and copies/renames the dev clips to data/dev_clips/devNN.mp4 with a README mapping table. Consequence to remember: adding clips and re-running can *reassign* which clips are dev/eval (dev = "shortest" is data-dependent).

scripts/run_pipeline.py — one clip through everything

Builds the two hooks and runs Pipeline:

- make_forecaster_hook(config) — reads config["forecaster"] to pick Kalman-CV or Seq2SeqForecaster(weights_path=config["seq2seq_weights"]) (new ablation switch; both share the same predict() interface). Forecasts in img or BEV space per forecast_space.
- make_risk_hook(config) — per frame: corridor (cached, dilated per actor class radius) → trajectory_collision_prob → EMA smoothing (skippable via ema_smoothing: false, new) → DecisionEngine.update() → events. Also writes context.active_threshold every frame for the dashboard.

scripts/mine_trajectories.py — build LSTM training data

For each train_traj clip: full-clip tracking → per-track resampling to 5 Hz buckets → sliding 30-point windows (10 history + 20 future, stride 1 s) → filters (drop tracks <6 s; drop windows with a consecutive jump >3x the median step = ID-switch artifact; drop near-static windows but KEEP 20% so "stationary stays stationary" is learnable) → normalize each window (translate last-history point to origin, rotate mean heading to +y, store the inverse transform) → one .npz shard per clip + aggregate stats.json. Resume logic: a clip is skipped iff its shard file exists; stats counters are *self-healed* each checkpoint by recounting actual shards on disk (so a stale/interrupted stats.json can't lie). Known quirk: a clip yielding 0 windows writes no shard → gets re-tracked every run (harmless).

scripts/eval_forecast.py — ADE/FDE table

Loads all shards, takes the same seeded 90/10 val split the training notebook used, runs BOTH forecasters over the same normalized windows, and reports ADE/FDE @1/2/4 s → eval/forecast_table.md. Δ --weights must be passed explicitly (default path ≠ notebook save path — it silently skips the LSTM row otherwise). Δ Numbers are only comparable within a run: growing the mined pool changes the val split, shifting BOTH models' absolute errors.

scripts/eval_anticipation.py — the headline eval (heavily extended)

Scores every clip in a split, two methods side by side:

- A3PS — first ALERT/VIRTUAL_BRAKE event time per clip.
 - Reactive-proximity baseline (new) — a no-forecasting ADAS: the first frame ANY actor is physically near the corridor (BEV < 2.0 m, else image < 8% of frame height — an exact Python port of the dashboard's reactive-ADAS marker in app.js, reusing point_in_dilated_polygon).
- Metrics per method: detection rate (alert *before* event_time_s), false-alarm rate (any alert on a negative), mTTA.
- Matched-subset mTTA (new, the fair comparison): per-method mTTA is confounded (each averages over its *own*, differently-sized true-positive set), so the report adds mTTA over only the positives BOTH methods anticipate — the sole number that can support "A3PS warns N s earlier."
 - --run (new behavior): processes missing clips with render=False (no video outputs) — resumable, much faster than step-6-style runs.
 - --sweep (new): caches each clip's per-frame max collision_prob once to eval/cache/<clip>.csv, then recomputes precision/recall at

base_threshold 0.40→0.90 (step 0.05) entirely in memory — an 11-point PR curve for the cost of zero extra GPU passes → eval/pr_curve_data.csv + eval/pr_curve.png. The sweep deliberately ignores the frames-to-confirm/cooldown debounce: it isolates the pure probability decision boundary with everything else held fixed.
Outputs: eval/anticipation.md, eval/anticipation_per_clip.csv.

scripts/run_ablations.py — Table II (new)

Runs eval_anticipation.py --run once per config variant, temporarily overwriting configs/default.yaml and always restoring it (restore after every variant + a .ablation_backup file + a finally block — a crash can never leave the config mutated). Variant rows in the paper's exact order: Kalman-CV / Seq2Seq-LSTM / Static threshold / Dynamic threshold / EMA off / EMA on. "Dynamic threshold" and "EMA on" are configurationally identical to the baseline, so they reuse its numbers (marked (= Kalman-CV) in the table) — only 4 of 6 rows cost GPU time. Each variant gets its own eval/ablation/<slug>/ clip cache; summaries are recomputed from each variant's per-clip CSV via the same tested compute_metrics → eval/ablation_table.md.

scripts/make_qual_figure.py — the paper's qualitative figure (new)

Finds one ALERT→VIRTUAL_BRAKE incident (matched by actor_id — per the state machine a brake always follows that same actor's alert), pulls 4 exact frames from raw.mp4 (just-before-ALERT, ALERT, BRAKE, just-after), renders each with the actual dashboard overlay code — a bare Pipeline.__new__ instance calling the real _draw_frame/_draw_prediction, so the figure can never drift from what the dashboard shows — and lays them out 2×2 at 300 DPI, captioning the BRAKE panel with the event's real explanation_template. Verified end-to-end on dashboard/clips/dev14.

scripts/collect_paper_stats.py — one-stop paper numbers

Read-only scan: config, hardware (incl. torch.cuda.is_available() — check this before quoting latency!), split sizes, per-stage latency aggregated across every meta.json (reads the per_stage_ms key the pipeline now writes), anticipation + forecast reports, exercised config variants → eval/paper_stats_summary.md. Missing inputs → NOT YET AVAILABLE + the command to produce them; it never invents numbers.

Others

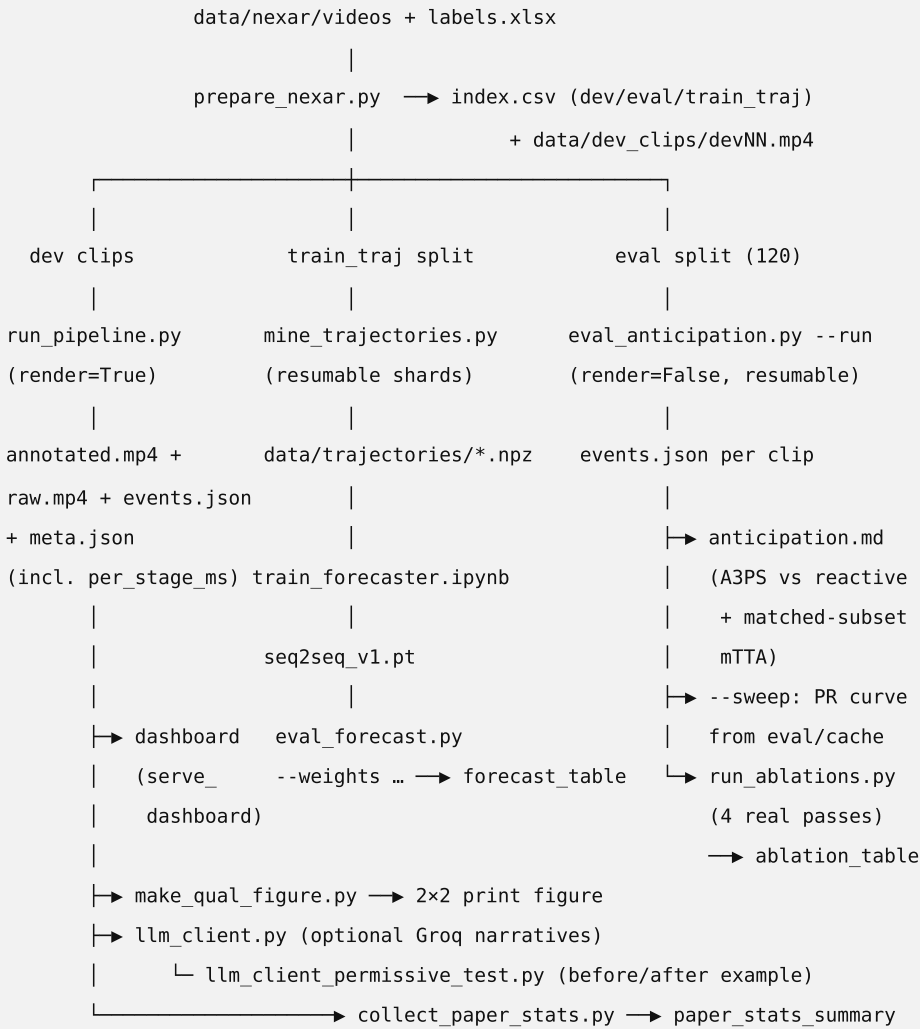
- scripts/serve_dashboard.py — static server for dashboard/ on :8000.
- scripts/update_manifest.py — regenerates dashboard/clips/manifest.json.
- scripts/verify_bev.py — sanity-checks a ground-plane calibration (velocities ~5-20 m/s, predictions ≥80% in frame).
- scripts/make_demo_clip.py / make_fake_events.py — synthetic demo data used to build the dashboard before the pipeline existed.

The dashboard — dashboard/{index.html, app.js}

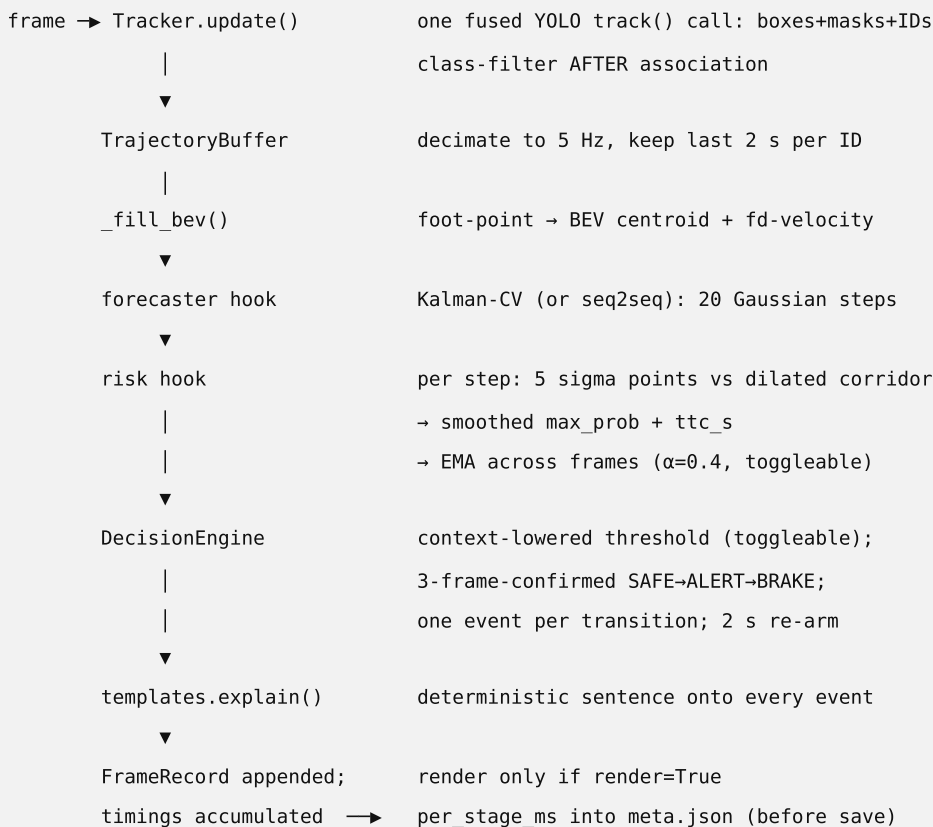
Pure static JS; reads ONLY the ClipResult schema. Canvas overlays (masks tinted by risk_level, trails, predicted paths, corridor), event log with explanation text, risk timeline with the active_threshold line stepping down under context flags, red banner/vignette 1.2 s after a VIRTUAL_BRAKE, and — important for the story — computeReactiveMarkers(): for each VIRTUAL_BRAKE it finds when a naive proximity ADAS would have reacted and draws ▼ (A3PS) vs ▽ (reactive) so the anticipation gap is visible per clip. This same logic is what eval_anticipation.py's baseline ports to Python.

Config — configs/default.yaml

Part 5 The end-to-end flow (with all new changes)



Per-frame flow inside one Pipeline.run():



The project's core claim, and where each piece of evidence comes from:

- `"It anticipates"` → `eval/anticipation.md` matched-subset mTTA vs the reactive baseline (`eval_anticipation.py`).
 - `"Each component earns its place"` → `eval/ablation_table.md` (`run_ablations.py`).
 - `"The threshold is a justified operating point"` → `eval/pr_curve.png` (`--sweep`).
 - `"The forecaster is sound"` → `eval/forecast_table.md` (`eval_forecast.py`).
 - `"It runs at a reportable speed"` → `per_stage_ms` in every `meta.json` (pipeline instrumentation), aggregated by `collect_paper_stats.py` — with the CPU-vs-GPU caveat stated explicitly.
 - `"It explains itself, faithfully"` → deterministic templates on every event + the constrained-vs-permissive LLM comparison (`llm_client_permissive_test.py`) + the qualitative figure (`make_qual_figure.py`).
-

Part 4 — Honesty notes baked into the design (do not lose these)

1. `perception = 0.0` in `per_stage_ms` is a reporting convention for a fused detector+tracker call, documented in-band via `per_stage_ms_note` — state it next to the latency table, don't let it read as "detection is free."
2. Latency numbers are hardware-bound. `collect_paper_stats` records whether CUDA was active; CPU-measured numbers must be labeled as such (~20-40x slower than GPU).
3. Matched-subset mTTA is the only defensible anticipation-gain number. Raw per-method mTTA can favor a worse method. Do not assume the sign — the 15-clip dev sanity run actually came out negative (A3PS ~2.9 s later than the naive baseline on the 4 shared clips), which is a real finding with real tuning knobs (`base_threshold`, `horizon_s`, `frames_to_confirm`), not a bug.
4. ADE/FDE are not comparable across different mined-pool sizes — the val split changes under both models. Compare within one run only. The old 51.24/51.64 "tie" and the newer 61.27/56.33 "LSTM wins @4s" are both true — on different val sets (archived in `eval/forecast_table_PREVIOUS.md` vs `eval/forecast_table.md`).
5. crosswalk/intersection flags are annotations, not detections — the docs say so explicitly; detecting them from imagery is future work.
6. The permissive-prompt "before" example is deliberately reproduced, not naturally occurring — disclose that in the paper; the script auto-restores the real narrative so the relaxed output can't leak.
7. `std_bev` holds pixels while `forecast_space: img` — a deliberate naming-stability choice until per-clip BEV calibration lands.